임베디드 리눅스 프로젝트

Jetson Smart Fan Controller

Linux Kernel Driver 기반 스마트 선풍기 제어 시스템

팀원 김호철 · 장태희 · 곽혜민 · 한소영

User Space 운전 정책 · Kernel Space 출력과 보호 정지

프로젝트 목표와 최종 구현

수동 입력과 온도로 풍속을 정하고, 상태를 표시하는 시스템



온도 기반 AUTO

장애 후 명시적 ON

0–5단계 출력

생존 알림·시간 제한

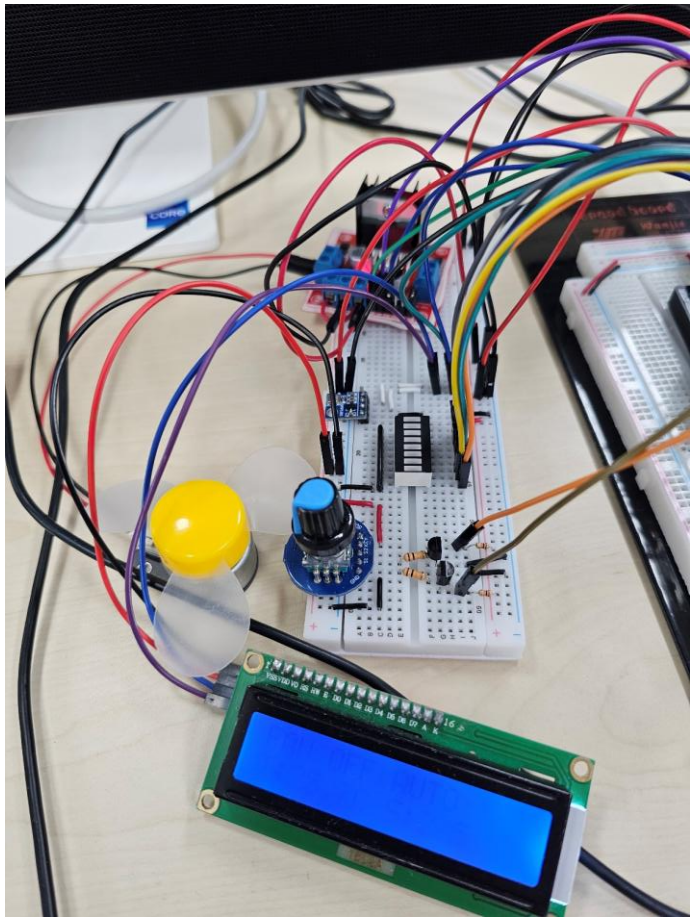
구현 범위: 운전·표시·보호 정지·운영 진단

학습 초점: 풍속 판단은 프로그램, 출력과 보호 정지는 Driver

출처: README.md · docs/architecture.md

실제 장치와 상태 표시

전체 구성과 LCD 표시 예시 · 입력·계측·구동·표시를 연결





LCD: 수동 운전 ON · 온도 20.3 °C · 3/5단계

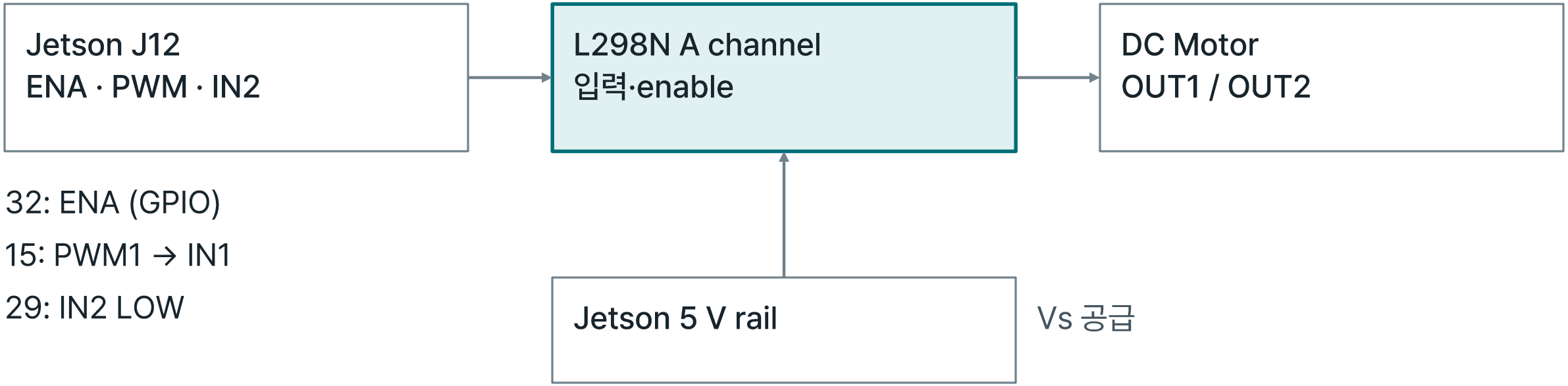
Jetson: 제어 프로그램 · L298N: 모터 구동

EC11: 수동 입력 · BMP180: 온도·기압 측정

출처: demo-thumbnail.jpg · 제공 사진 lcd-manual-status.jpg · hardware.md

신호 경로와 모터 전원 경로

ENA는 구동 허용, PWM은 풍속 조절 · 모터 전원은 5 V



32: ENA (GPIO)

15: PWM1 → IN1

29: IN2 LOW

공유 I²C bus7 · 100 kHz

BMP180 0x77 · LCD 0x27

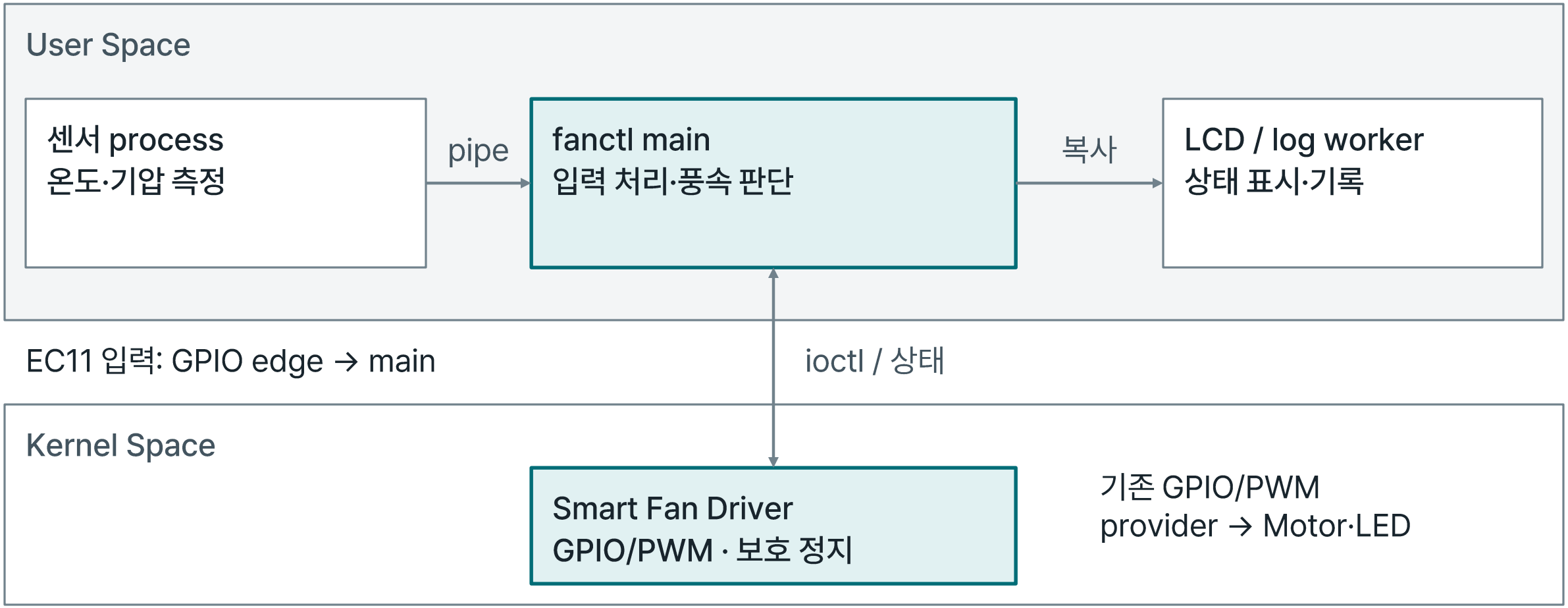
제어 신호의 역할

ENA: 켜기·끄기 · IN1: PWM 풍속
IN2: LOW 고정 · 회전 방향 유지

출처: docs/hardware.md · dts/smartfan.dts

사용자 프로그램과 커널의 역할

User Space는 풍속 판단 · Kernel Space는 출력과 보호 정지



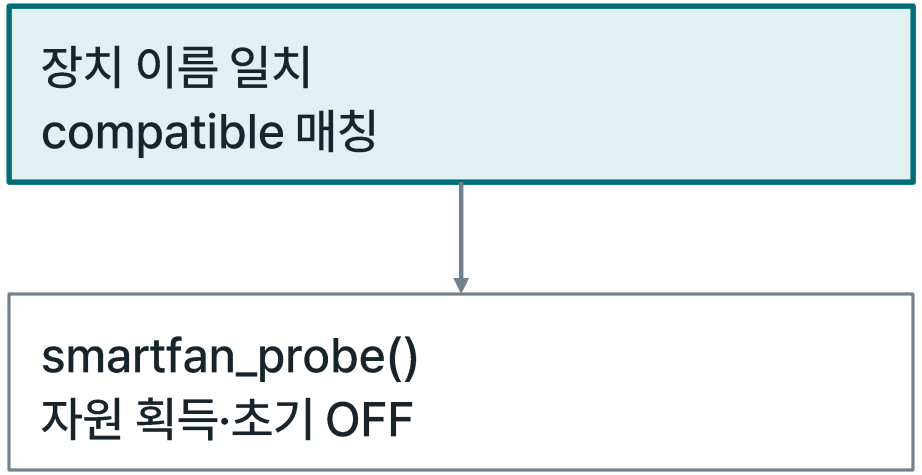
출처: app/main.c · docs/architecture.md · driver/smartfan.c

장치 설정과 Driver 연결

Device Tree의 장치 이름과 자원을 Driver에 연결

```
smartfan {
  compatible = "edu,jetson-smartfan";
  pwms = <&pwm1 0 4000000>;
  pwm-names = "motor";
  lease-timeout-ms = <2000>;
  max-on-ms = <30000>;
};
```

DT 핵심 행 발췌 · GPIO/pinctrl 속성 생략




```
devm_gpiod_get()
devm_pwm_get()
```

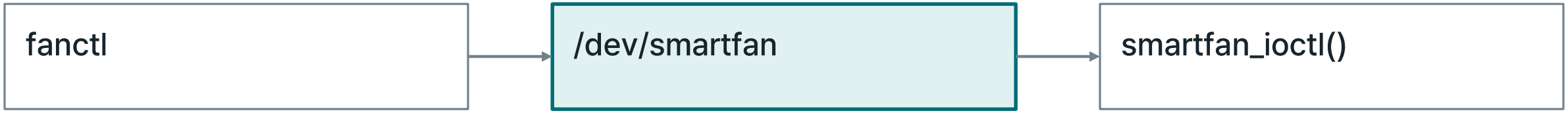
프로젝트 전용: Smart Fan Driver · 재사용: Linux GPIO/PWM Driver

probe에서 자원을 얻고 초기 출력을 OFF로 설정합니다.

출처: dts/smartfan.dts · smartfan_probe() · of_match_table

ioctl로 제어 요청과 상태 전달

장치 파일 /dev/smartfan을 통해 요청하고 결과를 확인




```
struct smartfan_auto_request {
    __u32 abi_version;
    __u32 level;
    __u32 rearm;
    __u32 reserved;
    __u64 sample_boottime_ms;
};
```

실제 UAPI · 고정 폭 ABI · 24 B

Driver의 요청 검사

1. 요청 데이터를 복사
2. 형식과 값의 범위 검사
3. 처리 후 상태 반환

SET 계열·HEARTBEAT: 제어

GET 계열: 상태·정지 사유

출처: include/smartfan_uapi.h · smartfan_ioctl() · fan-device.c

PWM 풍속과 LED 단계 표시

duty는 구동 신호의 ON 비율 · LED 칸 수로 선택 단계를 표시

단계	0	1	2	3	4	5
duty	0%	60%	70%	80%	90%	100%
LED	0	2	4	5	7	8

정지할 때는 구동 허용 신호부터 차단



PWM 250 Hz · 저속 기동 시 200 ms 동안 100% duty

단계 3은 LED 5칸에 대응합니다.

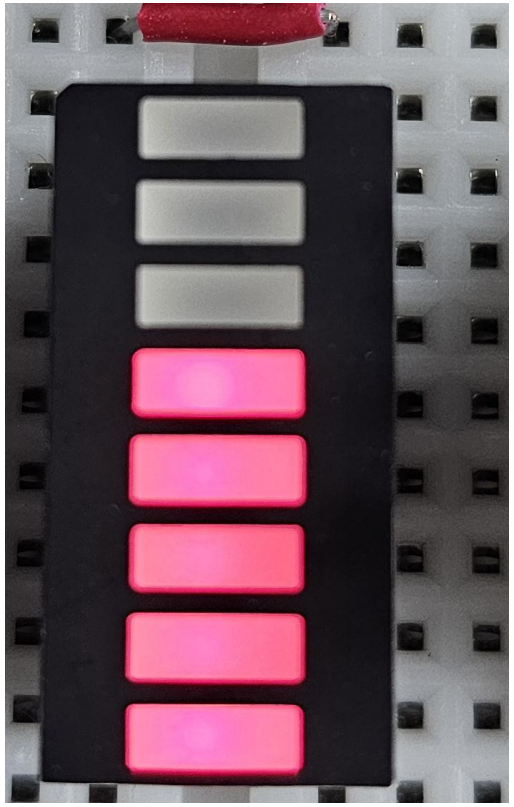


사진: LED 5칸 점등

출처: smartfan_level_duty() · smartfan_stop_locked() · 제공 사진 led-bar-output.jpg

프로그램 중단과 장시간 구동의 보호 정지

설정 기준: 생존 알림 250 ms · 유효 기간 2초 · 연속 구동 30초

생존 알림

250 ms

main의 기본 전송 간격
heartbeat 전송

알림 유효 기간

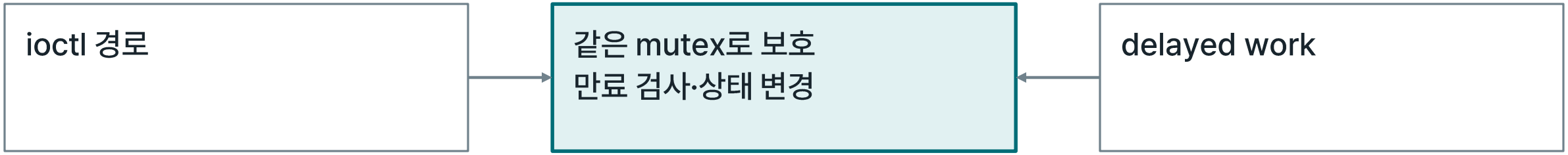
2 초

heartbeat 중단 시 만료
만료 → ENA 차단

연속 구동 상한

30 초

연속 구동 상한
반복 ON으로 연장 불가



명령 처리와 시간 만료 검사가 같은 상태를 동시에 바꾸지 않도록 합니다.

출처: smartfan_expire_locked() · smartfan_schedule_locked() · app/main.c

장치 종료와 절전 때의 출력 처리

구현 흐름: 먼저 OFF, 이후 접근 차단 · 복귀 후에도 OFF 유지

장치 정리 · remove

모터 OFF
새 자원 접근 차단

↓

장치 등록 해제
예약 작업 취소

열린 파일의 상태 객체는 참조가 끝날 때 정리

절전과 복귀 · suspend / resume

절전 진입
모터 OFF · 자동 재시작 금지

↓

복귀 후 OFF 유지
다시 켜려면 명시적 ON

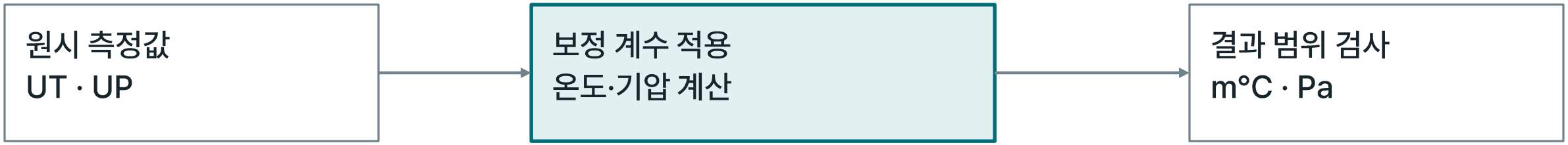
프로그램의 이전 요청으로 다시 켜지지 않도록 처리

출처: smartfan_open/release/remove/shutdown/suspend/resume()

BMP180에서 온도·기압을 얻는 과정

센서 사양을 확인해 변환 대기 시간과 보정 계산에 반영

사양에서 확인한 조건	코드에 반영한 처리
센서 ID와 보정 계수	ID 0x55 확인 · 보정 계수 22 B 읽기
온도 변환 최대 4.5 ms	5 ms 대기 후 완료 여부 확인
기압 변환 최대 25.5 ms · OSS=3	26 ms 대기 후 완료 여부 확인



센서와 LCD는 I²C bus7을 공유하며 100 kHz로 통신합니다.

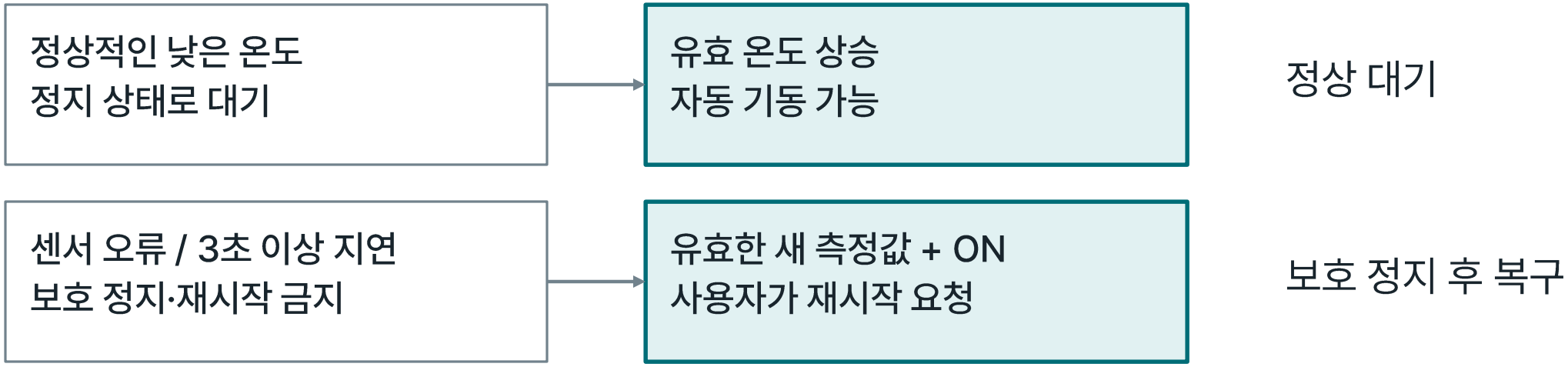
출처: app/bmp180.c · Bosch BMP180 DS000-09 §3.3–3.5, §4–5

온도 정책과 AUTO 재시작 조건

온도가 오르면 자동 구동 · 센서 오류 후에는 유효 값과 ON 필요

상승 경계	24 °C	26 °C	28 °C	30 °C	32 °C
목표 단계	1	2	3	4	5

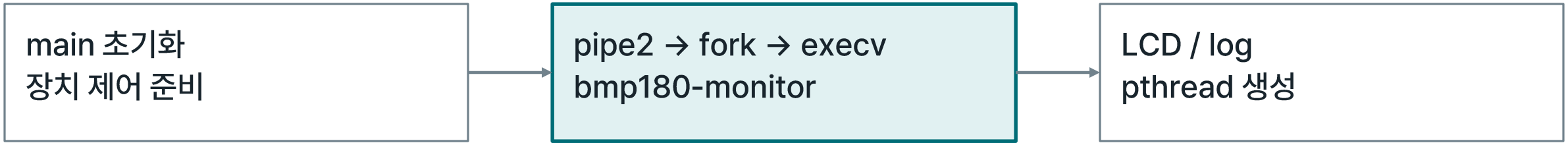
하강: 각 경계 − 1 °C 미만일 때 단계 감소



출처: auto_temperature_level() · controller_auto_tick() · smartfan_uapi.h

센서 대기가 제어를 막지 않도록 분리

센서는 별도 Process에서 측정 · main은 입력과 생존 알림 처리



초기화 실행 순서: fork는 Worker thread 생성 전에 수행

센서 process

센서 읽기·보정 · pipe로 결과 전달

main process

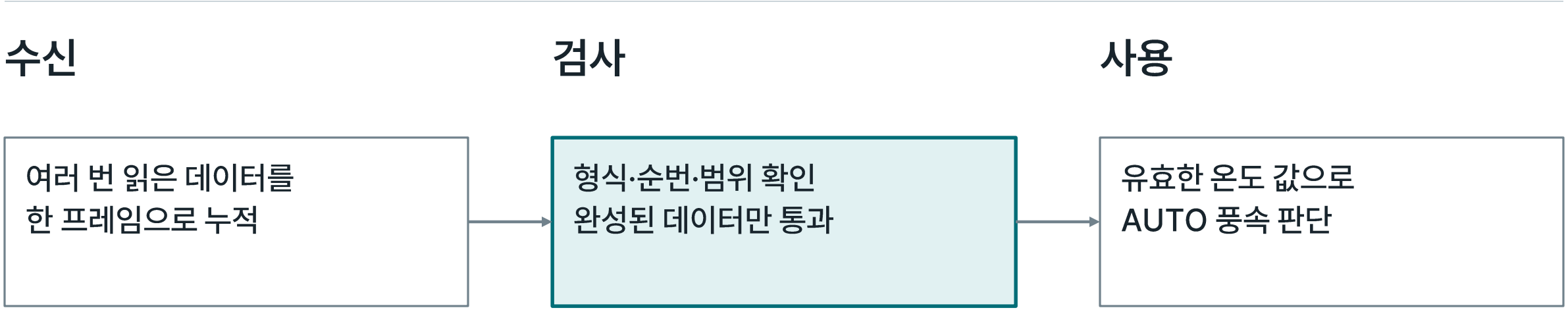
입력 처리 · 결과 수신 · 종료 회수

부모는 센서 Process의 시작과 종료를 관리하고 waitpid로 결과를 회수합니다.

출처: sensor_start() · bmp180-monitor.c · app/main.c

센서 데이터를 확인한 뒤 제어에 사용

Pipe IPC · 온도 값의 완성 여부, 유효성, 최신성을 각각 검사



완전한가?

32 B가 모였는지 확인

유효한가?

형식·순번·온도 범위 검사

최신인가?

측정 후 3초 미만인지 확인

출처: sensor.h · sensor_next() · bmp180-monitor::emit() · tests/test_sensor.c

여러 입력을 poll로 모아 처리

명령·노브·센서 입력을 main에서 처리해 운전 상태를 일관되게 변경

poll 대상	처리 목적
stdin / self-pipe	명령 입력 / 종료 신호 전달
GPIO edge fd	노브 두 채널의 신호 변화
Sensor pipe fd	센서 측정값·종료·오류 수신

회전 방향을 구분하는 신호 순서
00 → 10 → 11 → 01 → 00

신호 누락을 감지하면
현재 핀 상태로 다시 맞춤

신호 변화 4번 → 노브 1단계

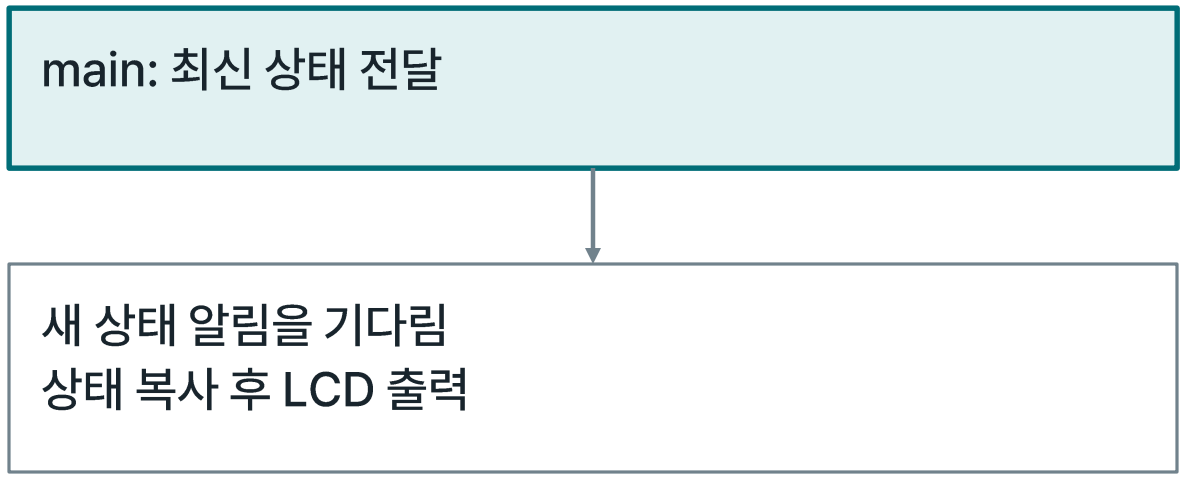
누락 전 이벤트를 버리고 기준 재설정

출처: app/main.c · encoder_read() · encoder_decoder_update()

화면 표시와 로그 기록을 별도 Thread로 처리

main은 필요한 데이터를 복사해 전달하고, Worker가 출력 작업 수행

LCD worker

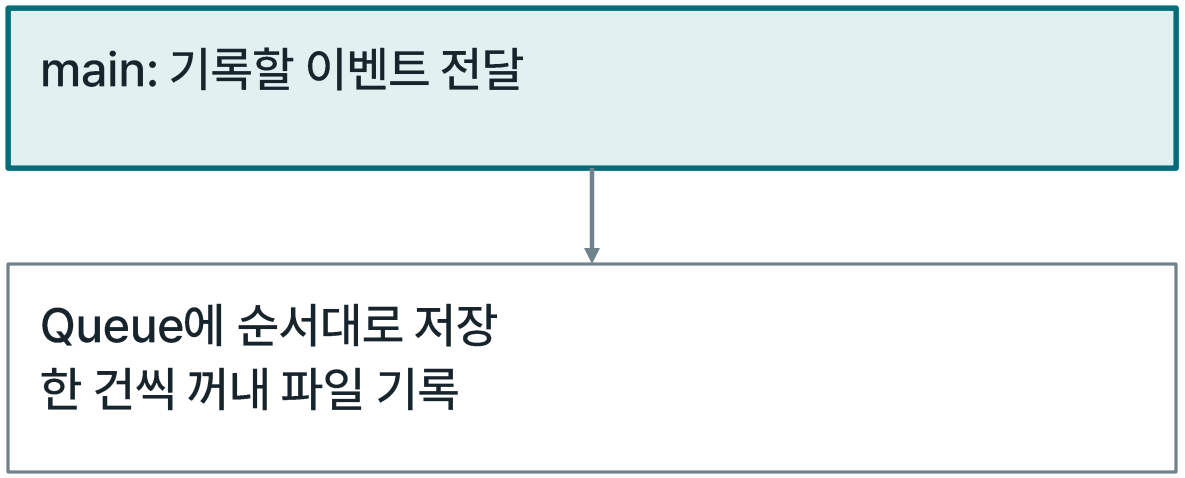


화면에는 최신 상태가 중요

Mutex 안에서 상태를 복사

I²C 출력은 잠금을 푼 뒤 수행

Event log worker · 선택형



로그에는 발생 순서가 중요

가득 차면 새 기록 제외·집계

종료 때 남은 기록 처리 후 합류

출처: app/lcd.c · app/event-log.c · docs/architecture.md

주요 문제 해결

ON인데 모터가 돌지 않았던 원인

증상 → pinmux 근거 → DT 수정 → 실물 회전 확인

증상·근거

ON 상태인데 모터 무회전

GPIO 자원 요청은 성공

실제 핀의 출력 설정이 맞지 않음

```
outputs {
  nvidia,tristate = <0>;
  nvidia,enable-input = <0>;
  nvidia,gpio-mode = <0>;
};
```

현재 DT 핵심 행 발췌



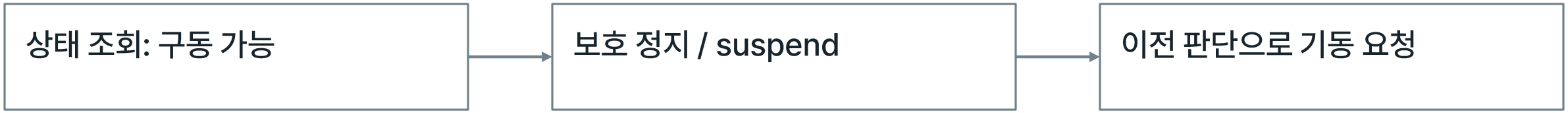
자원 획득과 물리 출력 조건을 각 계층에서 확인했습니다.

출처: docs/development.md · dts/smartfan.dts · scripts/prepare-dt.py

주요 문제 해결

상태 확인과 기동 사이의 경쟁 조건

조회 뒤 상태가 바뀔 수 있으므로, 실제 출력 직전에 다시 확인



문제: 조회 결과가 실제 출력 변경 시점의 상태를 보장하지 못함

```
if (!smartfan_auto_rearm_allowed(
        fan->auto_inhibited, auto_request.rearm)) {
    ret = -ECANCELED;
    break;
}
```

같은 Kernel mutex 안에서

장치가 동작 가능한지 확인

측정값이 최신인지 확인

재시작 허용 여부 확인 후 출력

소프트웨어 회귀: 상태 조회 뒤 보호 정지를 주입해 기동 거부 확인

출처: smartfan_ioctl() · controller_auto_tick() · tests/test_auto_guard.c

주요 문제 해결

센서를 종료한 뒤 통신 통로 정리

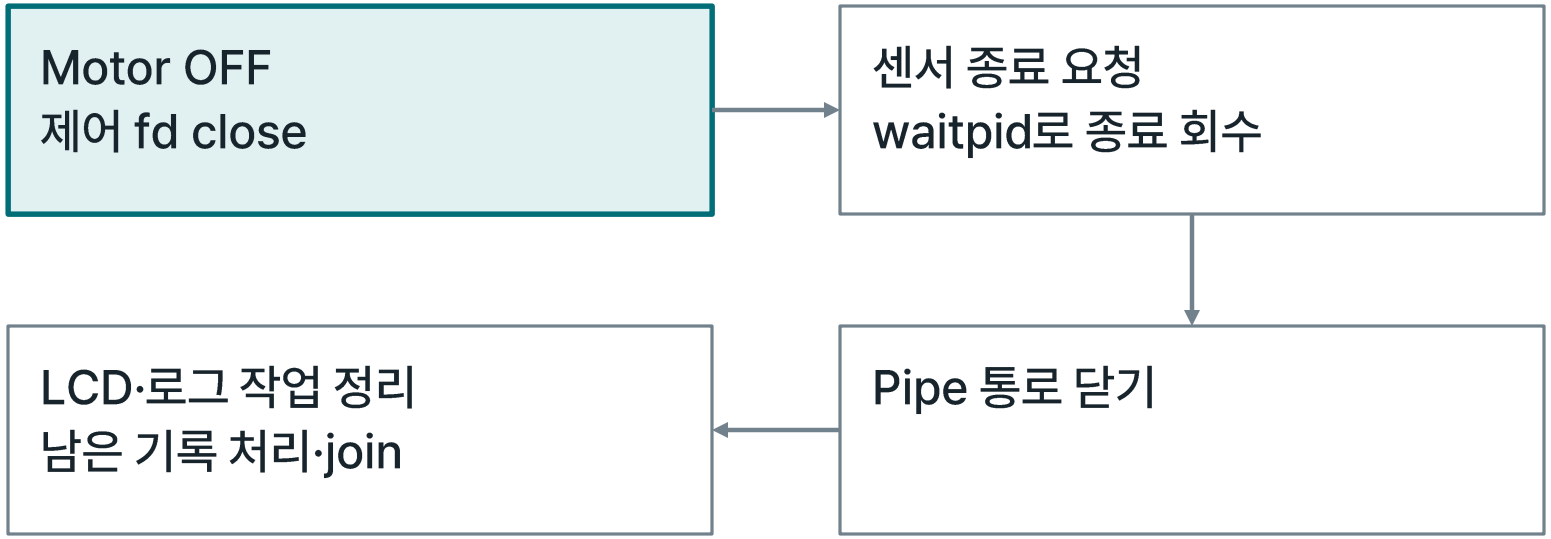
센서가 데이터를 쓰는 동안 수신 통로를 먼저 닫아 발생한 종료 오류

재현된 증상

수신 통로를 먼저 닫음

센서 write 중 SIGPIPE

수정한 종료 순서



검증: 통로가 꽉 찬 상태의 종료 · 종료 요청 무시 시 강제 회수

모터 정지 → 센서 종료 확인 → 통신 통로 정리

출처: sensor_stop() · tests/test_process.c · app/main.c

실제 장치 통합 시연

저장소 원본 영상 · 53.7초 · 클릭하여 재생



관찰할 흐름

speed 5 → on

엔코더 조작 → off

mode auto → on

손의 온기로 온도 상승

status 확인

기존 통합 동작 기록

모터·LED·온도·상태를
함께 관찰

출처: docs/assets/demo.mp4 · 제공된 시연 순서

확인한 결과와 다음 검증

실물 동작과 소프트웨어 검증 결과 · 세부 조건은 부록에 정리

검증 방법	확인한 내용
실물 동작 확인	모터·노브·온도 센서·LED·LCD 동작 AUTO·생존 알림 만료·시간 제한·종료 정지 보고
소프트웨어 검증	앱·커널 모듈·DT 빌드 통과 기록 정상/오류 경로 회귀 · 정적 분석 · 동시성 검사
다음 검증	최종 종료 수정 후 실물 재확인 · 실제 system suspend 전류·전압 여유·RPM·발열·장기 반복 측정

첨부 사진에서 LCD 상태 표시와 LED 5칸 점등을 확인했습니다.

출처: docs/review.md (2026-10-12) · tests/ · 제공된 LED/LCD 사진

Linux 개념의 통합과 후속 검증

풍속 판단·출력 제어·느린 작업을 나누고 실제 장치에 연결

Driver 경계	ioctl로 요청 전달 · Driver가 출력과 보호 정지 담당
Process / Thread	센서 대기·화면 표시·로그 기록을 제어 흐름에서 분리
오류·수명 관리	출력 직전 상태 재검사 · 정지부터 수행하는 종료 순서

다음 검증: 종료·suspend 실물 회귀 → 전원·RPM·발열·장기 반복 측정

핵심: 각 실행 흐름의 역할과 데이터를 주고받는 경계를 명확히 구성

출처: docs/architecture.md · docs/development.md · docs/review.md

J12 핀·인터페이스 대응표

질문 대응용 · 물리 header 번호, GPIO offset, DT 식별자를 구분

기능	J12 물리 핀	GPIO / 인터페이스
ENA / IN1 / IN2	32 / 15 / 29	41 PG.06 / PWM1 PN.01 / 105 PQ.05
EC11 S1 / S2	12 / 38	50 PH.07 / 52 PI.01 · 채널별 NPN
I²C SDA / SCL	3 / 5	bus7 · 100 kHz · BMP180 0x77 / LCD 0x27
LED 1–4	13 / 16 / 18 / 22	122 / 126 / 125 / 123
LED 5–8	33 / 35 / 37 / 40	43 / 53 / 124 / 51

3.3 V: BMP180·EC11·LCD · 5 V: L298N Vs · 공통 GND · ENA cap 제거

출처: docs/hardware.md · dts/smartfan.dts

ioctl 명령과 ABI 계약

고정 폭 필드·reserved 검사·구조 크기 검증 · 기존 ioctl 번호 유지

명령 / 번호	방향·구조 크기	계약
SET 1 / HEARTBEAT 2 / GET 3	W 16 B / 없음 / R 32 B	ON/OFF · lease 갱신 · 정지 사유
SET_SPEED 4 / GET_SPEED 5	W 16 B / R 32 B	0–5단계 · duty·capability
SET_LEDS 6 / GET_LEDS 7	W 16 B / R 16 B	test는 Motor OFF에서만 허용
SET_AUTO 8	W 24 B	level·rearm·BOOTTIME sample timestamp

open 단일 owner · SET_AUTO 차단: ESTALE / ECANCELED / EHOSTDOWN

copy 오류 EFAULT · 값 오류 EINVAL · unknown cmd ENOTTY · remove 후 ENODEV

출처: include/smartfan_uapi.h · smartfan_probe() · smartfan_ioctl()

BMP180 보정·AUTO 정책의 회귀 근거

Datasheet reference vector와 실제 정책 경계 테스트

Bosch 보정 예제 · OSS=0

입력: UT 27898 · UP 23843

결과: 15.000 °C · 69964 Pa

Reference vector · 실물 측정값과 구분

AUTO 경계 실제 코드

```
while (level && temperature_mc <
        thresholds[level - 1] - 1000)
    --level;
```

핵심 하강 조건 발췌

Endian·signed 계수 · malformed data · 분모·범위 오류 검사

정책 예: 24.000 °C → 단계 1 · 23.000 °C 유지 · 22.999 °C → 0

출처: tests/test_bmp180.c · auto_temperature_level() · Bosch DS000-09

회귀 테스트와 실물 재확인 항목

소프트웨어가 보호하는 계약과 장비에서 확인할 동작을 연결

소프트웨어 회귀	검증하는 계약	실물 후속 확인
AUTO ioctl stub	GET 뒤 stop·stale·명시적 rearm	STOP/CONT·fault/recovery 조합
실제 fork / pipe fixture	full pipe·SIGKILL 회수·parent death	최종 종료 후 Motor·Process 정리
LCD/Log 스트레스 + TSan	일관된 snapshot·queue 동기화	I²C 오류와 표시 실패 분리
DT offline 제안·installer check	base 선택·provider·적용 DT 비교	system suspend 실물 provider 복원 확인

```
make test · make analyze · make sanitize · make thread-check · make module W=1
```

출처: tests/test_auto_guard.c · test_process.c · test_lcd_concurrency.c · review.md